

COMSATS University Islamabad

Attock Campus



Department Of Computer Science

<i>Course:</i>	<i>Information Security</i>
<i>Instructor:</i>	<i>Ms: Ambreen Gul</i>
<i>L_Assignment</i>	<i>04</i>
<i>Due Date:</i>	<i>8th May, 2026</i>

Student Details

<i>Registration No:</i>	<i>FA24-BSE-030</i>
<i>Name:</i>	<i>Sayeda Aliza Hashmi</i>

Simple RSA Implementation.

1. Code Implementation – Line-by-Line Explanation

Imports

```
import hashlib
import random
```

hashlib provides the SHA-256 hash function used to produce a fixed-size digest of any message. random is used to generate large random prime numbers and a reproducible seed for key generation.

Miller-Rabin Primality Test – is_prime(n, k=5)

```
def is_prime(n, k=5):
    if n < 2: return False
    if n in (2, 3): return True
    if n % 2 == 0: return False
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1: continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1: break
        else: return False
    return True
```

The function first rejects trivially composite numbers (less than 2, even numbers). It then writes $n-1$ as $2^r * d$ (by repeatedly halving). For k independent witnesses a , it performs the Miller-Rabin test: if none of the squaring iterations reach $n-1$, the number is composite. Passing all k rounds gives a probabilistic primality certificate.

Prime Generation – generate_prime(bits=24)

```
def generate_prime(bits=24):
    while True:
        p = random.getrandbits(bits)
        if p >= 2 and is_prime(p):
            return p
```

Generates random integers of the specified bit length and tests each with `is_prime` until a probable prime is found. Using 24-bit primes keeps the demonstration fast while still illustrating RSA principles.

Extended GCD and Modular Inverse

```
def extended_gcd(a, b):
    if a == 0: return b, 0, 1
    g, x1, y1 = extended_gcd(b % a, a)
    return g, y1 - (b // a) * x1, x1
```

```
def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1: return None
    return x % m
```

extended_gcd implements the Extended Euclidean Algorithm recursively, returning the GCD and the Bezout coefficients (x, y) satisfying $a*x + b*y = \text{gcd}(a,b)$.

mod_inverse uses those coefficients: if $\text{gcd}(a, m) = 1$, then x from the equation is the modular inverse of a modulo m. If gcd is not 1, no inverse exists and None is returned.

Key Generation – generate_keys()

```
def generate_keys():
    random.seed(123)          # reproducible keys
    p = generate_prime(24)
    q = generate_prime(24)
    while q == p: q = generate_prime(24)
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537
    d = mod_inverse(e, phi)
    return (e, n), (d, n)
```

The seed makes key generation reproducible for testing. Two distinct 24-bit primes p and q are generated. $n = p*q$ is the RSA modulus. $\phi = (p-1)*(q-1)$ is Euler's totient. $e = 65537$ is the standard public exponent (a Fermat prime, balancing security and encryption speed). The private exponent d is the modular inverse of e modulo phi, satisfying $e*d \equiv 1 \pmod{\phi}$.

Returns a public key tuple (e, n) and a private key tuple (d, n).

Hash Function – hash_message(msg)

```
def hash_message(msg: str) -> int:
    return int.from_bytes(hashlib.sha256(msg.encode()).digest(), 'big')
```

Encodes the message to UTF-8 bytes, computes the SHA-256 digest (32 bytes), and interprets those bytes as a big-endian integer. This produces a 256-bit integer that uniquely represents the message content.

Sign and Verify

```
def sign(msg: str, priv_key):
    d, n = priv_key
    h = hash_message(msg)
    return pow(h % n, d, n)    # h % n ensures we sign inside the modulus

def verify(msg: str, signature: int, pub_key) -> bool:
    e, n = pub_key
    h = hash_message(msg)
    recovered_hash = pow(signature, e, n)
    return recovered_hash == (h % n)
```

sign: Extracts d and n from the private key, hashes the message, reduces the hash modulo n (because SHA-256 produces a 256-bit value but n is only ~48 bits), then computes the signature as $\text{pow}(h \% n, d, n)$ — the RSA private-key operation.

verify: Applies the public-key operation $\text{pow}(\text{signature}, e, n)$ to recover the hash, then compares with a fresh hash of the message. If any bit of the message changed, the hashes differ and verification fails.

Main Execution (Task 1)

```
pub, priv = generate_keys()
sig = sign(original, priv)
valid = verify(original, sig, pub)          # True
valid_tampered = verify(tampered, sig, pub) # False
```

Creates keys, signs the message, verifies the original (should be True), then verifies a one-character tampered version (should be False), demonstrating that the signature correctly detects modifications.

2. Security Analysis – Task 1

RSA Security Basis: The security of RSA relies on the integer factorisation problem. Given the public modulus $n = p \cdot q$, recovering the private exponent d without knowing p and q requires factoring n , which is computationally infeasible for large n (2048+ bits in production).

Hash Integrity: SHA-256 is a cryptographically secure hash function with strong preimage and collision resistance. Any modification to the message — even a single character — produces a completely different hash value (avalanche effect), causing signature verification to fail.

Non-repudiation: Only the holder of the private key d can produce a valid signature. The public key e allows anyone to verify it, but not to forge signatures, achieving non-repudiation.

Hash Reduction Limitation: Because this demonstration uses 24-bit primes, n is only ~48 bits while SHA-256 produces 256-bit hashes. The reduction $h \% n$ is a simplification not suitable for production — it discards upper bits and could theoretically allow collisions. In real RSA-PSS or PKCS#1 v1.5, proper padding schemes ensure the hash fits within the modulus without truncation.

Randomness and Key Size: The `random.seed(123)` makes the demo reproducible but entirely predictable — a serious vulnerability in practice. Real implementations use cryptographically secure random number generators (secrets module in Python) and prime sizes of at least 1024 bits per prime (2048-bit modulus minimum).

Timing Attacks: Python's `pow(base, exp, mod)` is not constant-time; side-channel attacks can potentially extract d by measuring execution time. Production libraries use blinding techniques to prevent this.

Create a Digital Signature.

1. Code Implementation – Line-by-Line Explanation

Imports

```
import hashlib
```

hashlib is the only import needed — this task uses fixed small primes (no random generation) and SHA-256 for hashing the message before signing.

Extended GCD and Modular Inverse

```
def extended_gcd(a, b):
    if a == 0: return b, 0, 1
    g, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return g, x, y

def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1: return None
    return x % m
```

Same Extended Euclidean Algorithm as Task 1 but with the return values stored in named variables (x, y) for clarity. The key mathematical guarantee is: $a * \text{mod_inverse}(a, m) \equiv 1 \pmod{m}$, which is essential for computing the RSA private exponent d from the public exponent e.

Key Generation – generate_keys()

```
def generate_keys():
    p, q = 61, 53
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 17
    d = mod_inverse(e, phi)
    return (e, n), (d, n)
```

Uses the classic textbook RSA example with the small primes $p=61$ and $q=53$, giving $n=3233$. $\phi(n) = 60 \cdot 52 = 3120$. The public exponent $e=17$ is chosen such that $\text{gcd}(17, 3120)=1$. The private exponent $d = \text{mod_inverse}(17, 3120) = 2753$.

This small-prime setup makes the mathematics traceable by hand and illustrates the structure of RSA clearly, though it offers no real security.

Hash Function – hash_message(message)

```
def hash_message(message: str) -> int:
    digest = hashlib.sha256(message.encode('utf-8')).digest()
    return int.from_bytes(digest, 'big')
```

Encodes the string message as UTF-8 bytes and computes the 32-byte (256-bit) SHA-256 digest. `int.from_bytes(..., 'big')` converts the raw bytes to a Python integer using big-endian byte order. This integer is what gets signed.

Sign – sign(message, priv_key)

```
def sign(message: str, priv_key: tuple) -> int:
    d, n = priv_key
    h = hash_message(message)
    # Signature = h^d mod n
    return pow(h, d, n)
```

Unpacks the private key into `d` and `n`. Computes the SHA-256 hash of the message as an integer. Applies the RSA private-key operation: $\text{signature} = h^d \bmod n$. Python's built-in `pow` with three arguments computes modular exponentiation efficiently using fast squaring.

Note: In this textbook version, `h` is not reduced modulo `n` before signing. This works only if $h < n$, which is not guaranteed with SHA-256 (256-bit hash vs. small `n` here). Task 1 addressed this with $h \% n$.

Verify – verify(message, signature, pub_key)

```
def verify(message: str, signature: int, pub_key: tuple) -> bool:
    e, n = pub_key
    decrypted_hash = pow(signature, e, n)
    original_hash = hash_message(message)
    return decrypted_hash == original_hash
```

Extracts `e` and `n` from the public key. Applies the RSA public-key operation to recover the hash: $\text{decrypted_hash} = \text{signature}^e \bmod n$. Independently recomputes the hash of the message being verified. Returns `True` only if both hashes match exactly, proving the signature was created with the corresponding private key and that the message is unaltered.

Main Execution (Task 2)

```
pub, priv = generate_keys()
original_msg = "Bob's transfer: $100"
sig = sign(original_msg, priv)          # 1. Sign
valid = verify(original_msg, sig, pub)  # 2. Verify original -> True
tampered = "Bob's transfer: $1000"
valid_tampered = verify(tampered, sig, pub) # 3. Verify tampered -> False
```

Generates the RSA key pair, signs the original financial message, then verifies it (should be `True`). Changes a single character ('\$100' to '\$1000') and verifies again with the same signature. The hash differs completely, so verification returns `False`, and the tampering-detected message is printed.

2. Security Analysis – Task 2

Digital Signature Guarantee: A digital signature provides three security properties simultaneously — authentication (the signer holds the private key), integrity (any message modification invalidates the signature), and non-repudiation (the signer cannot later deny creating it).

SHA-256 Avalanche Effect: Changing 'Bob's transfer: \$100' to 'Bob's transfer: \$1000' alters only one character, yet the SHA-256 hash changes completely and unpredictably. This demonstrates the avalanche effect: approximately 50% of the hash bits flip for any single-bit change in input.

Textbook RSA Limitation: This implementation uses raw RSA without padding (PKCS#1 v1.5 or PSS). Textbook RSA signatures are vulnerable to existential forgery — an attacker can craft valid signatures for arbitrary messages without knowing d , by exploiting the multiplicative structure of RSA. Real-world implementations mandate padding schemes.

Key Size: $p=61$ and $q=53$ give $n=3233$, which is trivially factorable. An attacker who factors n immediately recovers $\phi(n)$ and then $d = \text{mod_inverse}(e, \phi)$. Production RSA uses moduli of at least 2048 bits.

Hash Truncation vs. Collision: Because $n=3233 \ll 2^{256}$, the hash almost certainly exceeds n , meaning $\text{pow}(h, d, n)$ effectively operates on $h \bmod n$. Two different messages whose hashes share the same residue modulo n would produce identical signatures — a practical collision risk at this tiny key size. Proper RSA signatures avoid this via padding that guarantees the padded hash is in $[0, n-1]$.

Determinism: This signature scheme is deterministic given the same key and message, which is actually a security feature for RSA (unlike ECDSA, which requires a random nonce per signature). Reuse of a random nonce in ECDSA is catastrophic; RSA avoids this risk by design.